LAPORAN TUGAS BESAR 1

IF3170 Inteligensi Artifisial

Pencarian Konfigurasi Pemuatan Paket Optimal pada Peti Kemas Truk Menggunakan Algoritma Local Search

Disusun oleh:

Kelompok G58 – ApertureScience

No	Nama	NIM
1	Daniel Putra Rywandi S	13524143
2	Ahmad Rinofaros M	13524138
3	Mirza Tsabita W	13524114

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB I DESKRIPSI PERSOALAN **2**
1.1 Latar Belakang 2
1.2 Variabel Permasalahan 2
1.3 Fungsi Objektif dan Batasan 3
1.4 Ketentuan Representasi dan Langkah Neighbor 3
1.5 Algoritma yang Diimplementasikan 3
BAB II PEMBAHASAN **4**
2.1 Representasi State 4
2.2 Objective Function 5
2.3 Pembangkitan State Awal dan Neighbor 6
2.4 Implementasi Algoritma Local Search 7
2.4.1 Hill-climbing 7
2.4.2 Simulated Annealing 8
2.4.3 Genetic Algorithm 9
2.5 Heuristik yang Digunakan 9
2.6 Visualisasi 10
2.7 Hasil Eksperimen 10
2.7.1 Hill-climbing 10
2.7.2 Simulated Annealing 11
2.7.3 Genetic Algorithm 13
2.8 Analisis Hasil Eksperimen 14
2.8.1 Kedekatan terhadap Global Optima 14
2.8.2 Perbandingan Hasil Antar Algoritma 15
2.8.3 Perbandingan Durasi 15
2.8.4 Konsistensi Hasil 16
2.8.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm 16
2.8.6 Analisis Tambahan 17
BAB III KESIMPULAN DAN SARAN **18**
3.1 Kesimpulan 18
3.2 Saran 18
BAB IV PEMBAGIAN TUGAS **19**
REFERENSI **20**
LAMPIRAN **21**
Lampiran A. Form Penggunaan Generative AI 21
Lampiran B. Tautan Repository 21
Lampiran C. Lampiran Lainnya 21

BAB I DESKRIPSI PERSOALAN

1.1 Latar Belakang

Keluarga Mejiro sedang berencana mengekspansi bisnisnya ke bidang logistik darat dan ingin melakukan uji coba berskala kecil terlebih dahulu untuk torasen. Sebagai teman yang baik, Mejiro McQueen menawarkan posisi driver truk uji coba kepada Special Week karena sudah sering menyetir truk di kampung halamannya di Hokkaido. Usaha baru ini bernama Mejiro Express dengan slogan “Lebih kilat dari Umamusume”. Karena Special Week sibuk berlatih untuk memenangkan lomba-lombanya, dia tidak bisa membuang waktu berlama-lama untuk menentukan pengaturan pengiriman yang optimal. Oleh karena itu, dia meminta kalian, toreena-toreena yang pintar dalam keinformatikawan, untuk membuat suatu program yang cerdas untuk menolongnya.

Persoalan ini merupakan varian dari three dimensional bin packing problem yang dikombinasikan dengan knapsack problem dari sekumpulan paket berbentuk balok dengan dimensi, berat, dan nilai yang beragam, perlu dipilih paket mana yang dimuat, di koordinat mana, dan dengan orientasi apa, sehingga total nilai muatan maksimum tanpa melanggar batasan peti kemas. Ruang pencarian persoalan ini sangat besar sehingga pencarian menyeluruh tidak praktis. Oleh karena itu, persoalan didekati menggunakan algoritma local search yang bekerja dengan memperbaiki satu state secara bertahap.

1.2 Variabel Permasalahan

Entitas dan atribut yang terlibat dalam persoalan dirangkum pada Tabel 1.1.

Entitas	Atribut	Keterangan
Package	id	Identitas unik paket
	Dimensions (w, l, h)	Lebar, panjang, dan tinggi paket
	Orientation	Sumbu yang sejajar dengan tiap sisi paket; terdapat enam kemungkinan orientasi
	Value	Nilai harga paket (satuan unit)
	Weight	Berat paket (satuan unit berat)
	IsFragile	Penanda paket pecah-belah
	ETA	Perkiraan waktu kedatangan paket (sisa hari)
Truck	Dimensions (w, l, h)	Ukuran peti kemas pada truk
	MaxCapacity	Kapasitas angkut maksimum peti kemas (satuan unit berat)

Tabel 1.1 Variabel permasalahan

1.3 Fungsi Objektif dan Batasan

Tujuan persoalan adalah memaksimalkan total nilai (value) paket yang berada di dalam peti kemas. Sebuah konfigurasi dikatakan valid apabila memenuhi batasan berikut:

1. **Paket tidak melayang.** Untuk paket i dengan sudut kiri-bawah-belakang (x_i, y_i, z_i) dan dimensi setelah orientasi (w_i, d_i, h_i), alasnya adalah himpunan titik $B_i = \{(x, y, z_i) \mid x_i \leq x < x_i + w_i, y_i \leq y < y_i + d_i\}$. Sebuah titik alas memiliki support jika $z_i = 0$ (berada di lantai peti kemas) atau terdapat paket lain j yang bagian atasnya tepat di bawah titik tersebut. Paket dikatakan melayang apabila tidak ada satu pun titik alasnya yang memiliki support.
2. **Paket tidak menembus dinding peti kemas.** Sebuah paket harus seluruhnya berada di dalam peti kemas atau seluruhnya berada di luar; tidak boleh sebagian di dalam dan sebagian di luar.
3. **Barang pecah-belah tidak boleh berada di bawah paket lain,** artinya paket fragile tidak boleh menjadi support bagi paket lain.
4. **Total berat tidak melebihi kapasitas.** Jumlah berat paket di dalam peti kemas tidak boleh melebihi MaxCapacity.

Selain itu, secara implisit dua paket di dalam peti kemas tidak boleh saling bertumpukan overlap.

1.4 Ketentuan Representasi dan Langkah Neighbor

Paket berbentuk kubus atau balok dengan koordinat bilangan bulat, dan tidak semua paket harus dimasukkan ke dalam peti kemas. State awal dibangkitkan secara acak. Pada setiap iterasi, langkah neighbor yang diperbolehkan adalah:

1. Menukar dua paket, baik yang berada di dalam maupun di luar truk.
2. Memindahkan sebuah paket ke koordinat yang berbeda.
3. Merotasi paket 90° terhadap sumbu x, y, atau z.

State akhir harus mampu menunjukkan keadaan seluruh paket di dalam maupun di luar peti kemas, keadaan peti kemas, serta nilai objective function pada state tersebut.

1.5 Algoritma yang Diimplementasikan

Algoritma local search yang diimplementasikan pada tugas ini adalah Hill-climbing, Simulated Annealing, dan Genetic Algorithm.

BAB II PEMBAHASAN

2.1 Representasi State

Model data berada di direktori src/models. Setiap paket dimodelkan sebagai objek Package yang immutable dan berisi id, dimensi (width, length, height), value, weight, is_fragile, serta eta. Peti kemas dimodelkan dengan kelas Truck yang berisi width, length, height, dan max_capacity. Kedua kelas memvalidasi bahwa dimensi bernilai positif dan bahwa berat maupun kapasitas tidak negatif.

Sebuah state adalah objek State yang menyimpan daftar PackagePlacement, satu untuk setiap paket. PackagePlacement memasangkan sebuah Package dengan dua atribut yang dapat berubah selama pencarian:

- position: koordinat bilangan bulat (x, y, z) sudut kiri, bawah, belakang paket di dalam peti kemas, atau None jika paket berada di luar truk.
- orientation: permutasi indeks sumbu (a, b, c) yang menentukan dimensi asli mana yang menjadi lebar, panjang, dan tinggi efektif. Keenam orientasi disimpan pada konstanta ORIENTATIONS, dan dimensi efektif dihitung oleh properti dimensions.

Dengan representasi ini, satu state sudah menunjukkan keadaan seluruh paket di dalam maupun di luar truk, keadaan peti kemas, dan nilai muatannya. Properti inside_placements dan outside_placements memisahkan paket di dalam dan di luar truk. total_weight dan total_value menghitung berat dan nilai muatan. Method clone() menyalin daftar placement supaya pembangkitan neighbor tidak mengubah state asal, objek Package tidak ikut disalin karena immutable.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
Package	id, width, length, height, value, weight, is_fragile, eta	objek Package	Data paket yang tidak berubah; properti dimensions mengembalikan (width, length, height)
Truck	width, length, height, max_capacity	objek Truck	Data dimensi dan kapasitas peti kemas
PackagePlacement	package, position, orientation	objek PackagePlacement	Posisi dan orientasi paket; dimensions memberi dimensi setelah orientasi
State	placements	objek State	Kumpulan placement seluruh paket beserta properti agregat dan clone()

2.2 Objective Function

Objective function utama diimplementasikan oleh calculate_objective di src/utils/objective.py. Nilainya adalah total value paket di dalam peti kemas dikurangi penalti pelanggaran batasan:

$$f(s) = \Sigma \text{ value}(p) \text{ untuk setiap } p \in \text{inside}(s) - P(s)$$

Fungsi get_violations di src/utils/constraints.py menghitung lima jenis pelanggaran. Fungsi violation_units menjumlahkan pelanggaran tersebut menjadi U(s), yaitu banyaknya pelanggaran pada state s. Pelanggaran kelebihan berat dihitung sebagai 1 ditambah selisih beratnya, sehingga kelebihan berat sekecil apa pun bernilai minimal 1.

Pelanggaran	Fungsi Pemeriksa	Cara Penghitungan	Bobot Penalti
Paket menembus dinding	is_within_bounds	Banyak paket di dalam truk yang melewati batas peti kemas	500 per paket
Paket tumpang-tindih	check_overlap	Banyak pasangan paket yang ruangnya beririsan pada ketiga sumbu	400 per pasangan
Paket melayang	is_supported	Banyak paket dengan z > 0 yang alasnya tidak bersentuhan dengan sisi atas paket non-fragile mana pun	300 per paket
Paket fragile tertimpa	check_fragile_violations	Banyak paket fragile yang memiliki paket lain di atasnya dengan proyeksi alas yang beririsan	350 per paket
Kelebihan berat	get_violations	Selisih total berat di atas max_capacity	25 per unit berat

Tabel 2.2 Jenis pelanggaran dan bobot penalti

Pemeriksaan support mengikuti definisi pada spesifikasi: sebuah paket tidak melayang jika minimal satu titik alasnya ditopang, yaitu jika z = 0 atau ada paket lain yang sisi atasnya (oz + oh) tepat sama dengan z dan proyeksi alasnya beririsan. Paket fragile tidak dihitung sebagai penopang. Karena itu, paket yang hanya bertumpu pada paket fragile dianggap melayang dan juga menimbulkan pelanggaran fragile. Dengan pendekatan penalti, pencarian tetap dapat melewati state tidak valid sebagai batu loncatan. Penalti kelebihan berat dihitung per unit berat sehingga perbaikan kecil tetap menaikkan objective.

Sebagai bonus, kelompok mengimplementasikan tiga objective function alternatif:

- objective_eta_priority: Σ value·(1 + 1/(eta + 1)), yang memprioritaskan paket dengan ETA kecil.
- objective_value_density: Σ 5·value / max(0.1, weight), yang memprioritaskan paket bernilai tinggi per satuan berat.
- objective_volume_efficiency: total value × (0.8 + 0.2 × rasio volume terpakai), yang memberi bonus untuk pemakaian ruang yang lebih penuh.

Setiap fungsi algoritma menerima parameter objective_func sehingga objective function dapat diganti tanpa mengubah kode algoritma.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
is_within_bounds	placement, truck	bool	True jika paket di luar truk atau seluruh bagiannya berada di dalam peti kemas
check_overlap	p1, p2	bool	True jika dua paket di dalam truk saling beririsan pada sumbu x, y, dan z
is_supported	placement, inside_placements	bool	True jika paket berada di lantai atau ditopang paket non-fragile tepat di bawahnya
check_fragile_violations	inside_placements	int	Banyak paket fragile yang tertimpa paket lain
get_violations	state, truck	dict	Rekap seluruh pelanggaran dan status is_valid
is_valid_state	state, truck	bool	True jika tidak ada pelanggaran
calculate_penalty	state, truck	float	Total penalti berbobot (Tabel 2.2)
calculate_objective	state, truck, penalized=True	float	Total value muatan dikurangi penalti
objective_eta_priority, objective_value_density, objective_volume_efficiency	state, truck, penalized=True	float	Objective function alternatif (bonus)

Tabel 2.3 Fungsi batasan dan objective function

2.3 Pembangkitan State Awal dan Neighbor

State awal dibangkitkan oleh create_random_state. Untuk setiap paket dipilih orientasi acak. Jika paket muat dalam orientasi itu, paket dimasukkan ke truk dengan peluang place_prob = 0,6 pada koordinat x dan y acak. Koordinat z bernilai 0 (lantai) dengan peluang 0,7 dan acak pada sisanya. Paket lainnya diletakkan di luar truk. State awal ini dapat mengandung pelanggaran, misalnya tumpang-tindih atau paket melayang, yang kemudian dikurangi oleh pencarian melalui penalti.

Ketiga langkah neighbor diimplementasikan di src/utils/moves.py:

4. **swap_move** memilih dua paket secara acak, baik di dalam maupun di luar truk, lalu menukar posisinya. Posisi hasil pertukaran dibatasi (clamp) sesuai dimensi paket barunya agar tidak menembus dinding. Menukar paket di dalam truk dengan paket di luar berarti mengeluarkan satu paket dan memasukkan paket lain ke tempatnya.
5. **relocate_move** memilih satu paket secara acak. Paket yang sedang di dalam truk dikeluarkan dengan peluang 0,2. Selain itu, jika paket muat, posisinya diubah dengan salah satu dari tiga mode: 40% ke koordinat acak di lantai; 30% ditumpuk di atas paket non-fragile acak (x dan y disejajarkan dengan paket alas, z sama dengan sisi atas paket alas), atau ke lantai bila tinggi peti kemas tidak cukup; dan 30% ke koordinat (x, y, z) acak. Paket yang tidak muat dalam orientasinya dikeluarkan.
6. **rotate_move** memilih satu paket, dengan peluang 0,75 dari paket yang ada di dalam truk, lalu mengganti orientasinya dengan salah satu dari lima orientasi lain. Posisinya kemudian dibatasi agar tetap berada di dalam peti kemas.

get_random_neighbor memilih salah satu dari ketiga move secara seragam. generate_neighbors membangkitkan sejumlah neighbor acak (bawaan 40) untuk varian Hill-climbing yang membandingkan beberapa kandidat.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
create_random_state	packages, truck, place_prob=0.6	State	Membangkitkan state awal secara acak dengan menempatkan sebagian paket di dalam peti kemas
swap_move	state, truck	State	Menukar posisi dua paket acak, baik yang berada di dalam maupun di luar truk
relocate_move	state, truck	State	Memindahkan satu paket acak ke koordinat baru di lantai, menumpuknya, atau mengeluarkannya
rotate_move	state, truck	State	Mengubah orientasi paket acak menjadi salah satu dari lima orientasi lainnya
get_random_neighbor	state, truck	State	Memilih satu aksi neighbor secara acak (swap, relocate, atau rotate) dan mengembalikan state baru
generate_neighbors	state, truck, num_neighbors=40	List[State]	Membangkitkan sejumlah neighbor acak untuk dibandingkan pada pencarian local search

Tabel 2.4 Fungsi pembangkitan state awal dan neighbor

2.4 Implementasi Algoritma Local Search

2.4.1 Hill-climbing

Varian Hill-climbing yang diimplementasikan pada tugas ini meliputi Steepest-ascent Hill-climbing, Hill-climbing with Sideways Moves, dan Random-restart Hill-climbing. Pada Steepest-ascent Hill-climbing, algoritma mengevaluasi seluruh neighbor kandidat yang dibangkitkan pada setiap iterasi dan memilih neighbor dengan nilai objective terbaik selama nilainya lebih tinggi dari state saat ini. Jika tidak ada neighbor yang lebih baik, pencarian berhenti di local optima. Pada varian dengan Sideways Moves, algoritma memperbolehkan perpindahan ke neighbor dengan nilai objective yang sama hingga batas maksimum tertentu (max_sideways=100) untuk keluar dari shoulder atau plateau. Pada Random-restart Hill-climbing, ketika pencarian terjebak di local optima, state dibangkitkan kembali secara acak hingga jumlah restart maksimum (max_restarts=10) tercapai, lalu solusi terbaik dari seluruh run diambil sebagai hasil akhir.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
steepest_ascent_hill_climbing	initial_state, truck, max_iterations=1000, num_neighbors=40	State, List[float]	Implementasi Steepest-ascent Hill-climbing yang memilih neighbor terbaik pada setiap iterasi
random_restart_hill_climbing	packages, truck, max_restarts=10, max_iterations=500	State, List[float]	Implementasi Random-restart Hill-climbing yang mengulang pencarian dari state awal acak baru

Tabel 2.4.1 Fungsi/kelas pada Hill-climbing

Potongan source code src/algorithms/hill_climbing.py:

```
def stochastic_hill_climbing(
    initial_state: State,
    truck: Truck | list[Truck],
    max_iterations: int = 500,
    num_candidates: int = 30,
    objective_func = calculate_objective
) -> HillClimbingResult:
    start_time = time.time()
    current = initial_state.clone()
    current_val = objective_func(current, truck)
    history = [current_val]

    iteration = 0
    while iteration < max_iterations:
        iteration += 1
        candidates = generate_neighbors(current, truck, count=num_candidates)
        uphill_moves = []

        for cand in candidates:
            cand_val = objective_func(cand, truck)
            if cand_val > current_val:
                uphill_moves.append((cand, cand_val))

        if uphill_moves:
            chosen_state, chosen_val = random.choice(uphill_moves)
            current = chosen_state
            current_val = chosen_val
```

```
            history.append(current_val)
        else:
            history.append(current_val)
            break

    duration = time.time() - start_time
    return HillClimbingResult(
        algorithm_name="Stochastic Hill Climbing",
        initial_state=initial_state,
        final_state=current,
        initial_value=history[0],
        final_value=current_val,
        iterations=iteration,
        duration=duration,
        history=history,
        metadata={"stop_reason": "local_optima" if iteration < max_iterations else
"max_iterations"}
    )
```

2.4.2 Simulated Annealing

Simulated Annealing diimplementasikan untuk memungkinkan pencarian melompati local optima dengan menerima state neighbor yang buruk ($\Delta E < 0$) berdasarkan nilai probabilitas penerimaan $p = e^{(\Delta E / T)}$. Nilai T diawali pada T_initial = 1000.0 dan diturunkan secara bertahap pada setiap iterasi menggunakan fungsi skedul pendinginan eksponensial $T(t) = T_{initial} \cdot \alpha^t$ dengan faktor pendinginan $\alpha = 0.995$. Algoritma berhenti apabila temperatur mencapai T_min = 0.001 atau iterasi mencapai max_iterations = 2000. Kejadian "stuck" di local optima didefinisikan ketika nilai objective tidak mengalami peningkatan signifikan (selisih < 0.01) selama 100 iterasi berturut-turut.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
simulated_annealing	initial_state, truck, initial_temp=1000.0, cooling_rate=0.995, min_temp=0.001	State, List[float], List[float]	Implementasi Simulated Annealing dengan penerimaan probabilitas dan pelacakan probabilitas penerimaan e^(ΔE/T)
cooling_schedule	temp, cooling_rate	float	Menghitung temperatur berikutnya menggunakan rumus T_next = T * alpha

Tabel 2.4.2 Fungsi/kelas pada Simulated Annealing

Potongan source code src/algorithms/simulated_annealing.py:

```
while iteration < max_iterations and temp > min_temp:
        iteration += 1
        neighbor = get_random_neighbor(current, truck)
        neighbor_val = objective_func(neighbor, truck)
```

```
        delta_e = neighbor_val - current_val
        exponent = max(-700.0, min(700.0, delta_e / max(1e-9, temp)))
        exp_val = math.exp(exponent)
        exp_history.append(exp_val)

        if delta_e > 0:
            prob = 1.0
            acceptance_prob_history.append(prob)
            current = neighbor
            current_val = neighbor_val

            if current_val > best_val:
                best_val = current_val
                best_state = current.clone()
        else:
            prob = min(1.0, exp_val)
            acceptance_prob_history.append(prob)

            if random.random() < prob:
                current = neighbor
                current_val = neighbor_val
            else:
                stuck_count += 1

        history.append(current_val)
        temp *= cooling_rate
        temperature_history.append(temp)

    duration = time.time() - start_time
    stuck_frequency = stuck_count / max(1, iteration)
```

2.4.3 Genetic Algorithm

Pada Genetic Algorithm, setiap individu (kromosom) direpresentasikan secara langsung oleh sebuah objek State yang berisi susunan lokasi dan orientasi seluruh paket. Fungsi fitness dihitung secara langsung dari fungsi objective terpenalti calculate_objective(state, truck). Seleksi orang tua menggunakan metode Tournament Selection dengan ukuran turnamen k = 3. Crossover dilakukan dengan Uniform Crossover pada tingkat placement paket, di mana setiap placement diambil dari salah satu orang tua dengan probabilitas 0.5. Mutasi dilakukan dengan menerapkan salah satu langkah neighbor (swap, relocate, atau rotate) pada individu dengan probabilitas mutasi mutation_rate = 0.2. Individu hasil crossover dan mutasi yang tidak valid diperbaiki secara tidak langsung melalui penalti pada fungsi fitness, mendorong evolusi secara alami menuju individu-individu yang valid.

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
genetic_algorithm	packages, truck, pop_size=50, generations=100, mutation_rate=0.2	State, List[float], List[float]	Implementasi Genetic Algorithm yang mengembalikan individu terbaik beserta riwayat fitness max dan rata-rata

Fungsi / Kelas	Parameter	Keluaran	Deskripsi
crossover	parent1, parent2, truck	State	Melakukan Uniform Crossover antara dua individu untuk menghasilkan anak baru

Tabel 2.4.3 Fungsi/kelas pada Genetic Algorithm

Potongan source code src/algorithms/genetic_algorithm.py:

```
def single_point_crossover(p1: State, p2: State) -> tuple[State, State]:
    n = len(p1.placements)
    if n < 2:
        return p1.clone(), p2.clone()

    point = random.randint(1, n - 1)
    c1_placements = [
        PackagePlacement(p.package, p.position, p.orientation, p.truck_idx)
        for p in (p1.placements[:point] + p2.placements[point:])
    ]
    c2_placements = [
        PackagePlacement(p.package, p.position, p.orientation, p.truck_idx)
        for p in (p2.placements[:point] + p1.placements[point:])
    ]
    return State(c1_placements), State(c2_placements)
```

2.5 Heuristik yang Digunakan

1. State awal acak yang valid. Paket hanya dicoba ditaruh di lantai atau di atas paket non-fragile, dan kandidat posisi yang melanggar batasan ditolak. Dengan begitu pencarian selalu dimulai dari state valid dan tidak menghabiskan iterasi awal untuk memperbaiki pelanggaran.

2. Penalti sebesar batas atas skor. Bobot penalti per pelanggaran sama dengan skor jika seluruh paket dimuat ditambah 1, sehingga state yang melanggar batasan tidak pernah lebih baik daripada state valid.

3. Mode relocate yang terarah. Sebagian besar relocate (70%) menaruh paket di lantai atau tepat di atas paket non-fragile, sehingga neighbor yang dihasilkan lebih sering tidak melayang. Sisanya (30%) memakai koordinat acak penuh untuk menjaga keragaman.

4. Rotasi diprioritaskan untuk paket di dalam truk. Rotasi paket di luar truk tidak mengubah objective, sehingga rotate_move memilih paket di dalam truk dengan peluang 0,75.

5. Clamping posisi. Setelah swap dan rotasi, posisi paket digeser agar tetap berada di dalam peti kemas, sehingga move tersebut tidak menghasilkan paket yang menembus dinding.

6. Neighbor lengkap dengan sampel relocate. Steepest-ascent dan Sideways Move mengevaluasi seluruh swap dan rotasi, sedangkan relocate hanya disampel karena satu paket dapat dipindahkan ke W × L × H koordinat di setiap truk.

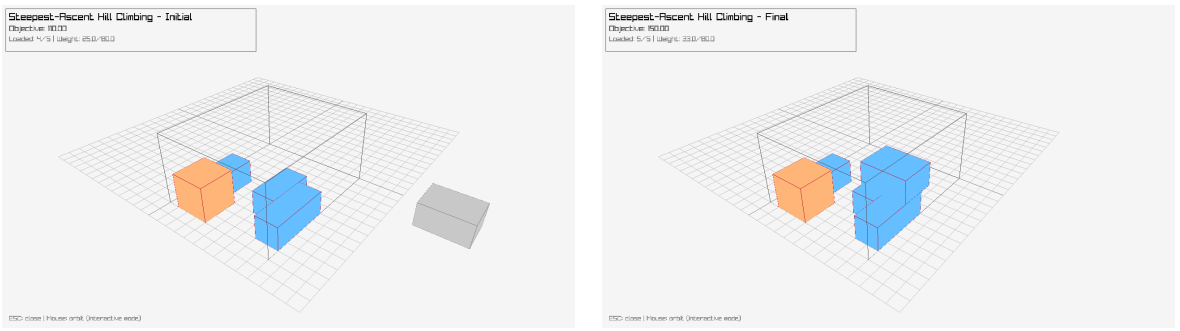
7. Elitisme pada Genetic Algorithm. Dua individu terbaik selalu dipertahankan sehingga solusi terbaik tidak hilang akibat crossover atau mutasi.

2.6 Visualisasi

Visualisasi program digunakan untuk menampilkan konfigurasi penempatan paket di dalam peti kemas truk dalam bentuk tiga dimensi. Setiap paket direpresentasikan sesuai dengan dimensi, posisi, dan orientasinya sehingga kondisi penempatan paket dapat diamati secara langsung.

Program menyediakan visualisasi *initial state* untuk memperlihatkan konfigurasi awal sebelum proses pencarian dan *final state* untuk menunjukkan hasil konfigurasi setelah algoritma dijalankan. Dengan membandingkan kedua state tersebut, pengguna dapat mengamati perubahan susunan paket yang terjadi selama proses optimasi.

Visualisasi diimplementasikan menggunakan pustaka Raylib dan mendukung penyimpanan hasil dalam bentuk gambar. Selain visualisasi tiga dimensi, perkembangan nilai *objective function* juga ditampilkan melalui grafik terhadap banyaknya iterasi.



Gambar 2.1 Contoh visualisasi state

2.7 Hasil Eksperimen

2.7.1 Hill-climbing

Pengujian algoritma *Steepest-ascent Hill-climbing* dilakukan sebanyak tiga kali menggunakan data sample.json dengan *random seed* 42, 43, dan 44. Setiap pengujian dimulai dari konfigurasi paket yang dibangkitkan secara acak.

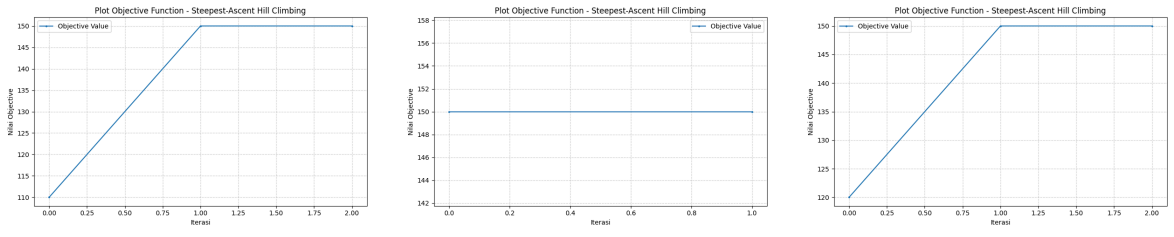
Pada setiap iterasi, algoritma mengevaluasi kandidat *neighbor* dan memilih konfigurasi dengan nilai *objective function* tertinggi apabila memberikan peningkatan terhadap state saat ini. Proses pencarian berhenti ketika tidak ditemukan *neighbor* yang lebih baik atau ketika batas iterasi tercapai.

Run	Objective Awal	Objective Akhir	Banyak Iterasi	Durasi (s)	Keterangan
1	110	150	2	0,0071	Local optima
2	150	150	1	0,0037	Local optima
3	120	150	2	0,0074	Local optima

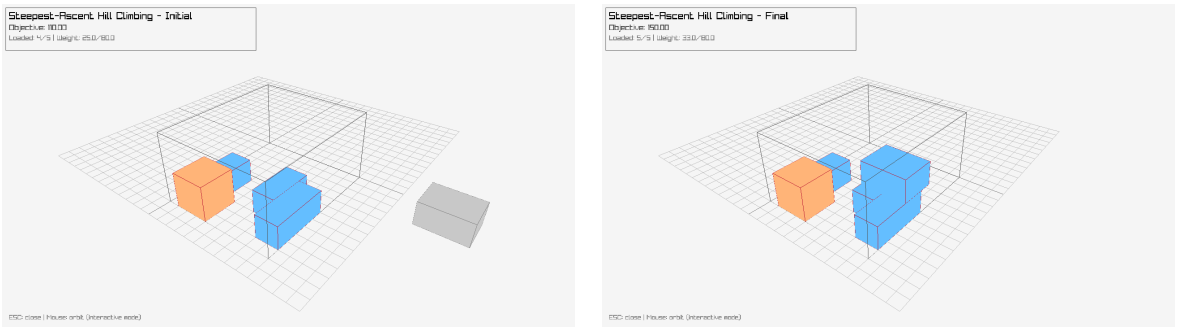
Tabel 2.2 Hasil eksperimen Hill-climbing

Berdasarkan Tabel 2.2, ketiga pengujian Steepest-ascent Hill-climbing menghasilkan nilai objective function akhir yang sama, yaitu 150, meskipun memiliki nilai awal dan jumlah iterasi yang berbeda. Pada pengujian pertama, nilai objektif meningkat dari 110 menjadi 150 dalam 2 iterasi. Pengujian kedua memperoleh nilai objektif 150 sejak awal dan berhenti setelah 1 iterasi, sedangkan pengujian ketiga meningkat dari 120 menjadi 150 dalam 2 iterasi.

Durasi pencarian masing-masing pengujian adalah 0,0071 detik, 0,0037 detik, dan 0,0074 detik. Seluruh pengujian berhenti karena tidak ditemukan neighbor yang memberikan peningkatan nilai objektif (local optima). Ketiga konfigurasi akhir memenuhi batasan pemuatan, dengan seluruh 5 paket berhasil dimasukkan ke dalam peti kemas dan total berat 33 dari kapasitas maksimum 80.



Gambar 2.2 Plot objective function terhadap iterasi (Hill-climbing, run 1–3)



Gambar 2.3 State awal dan akhir Hill-climbing

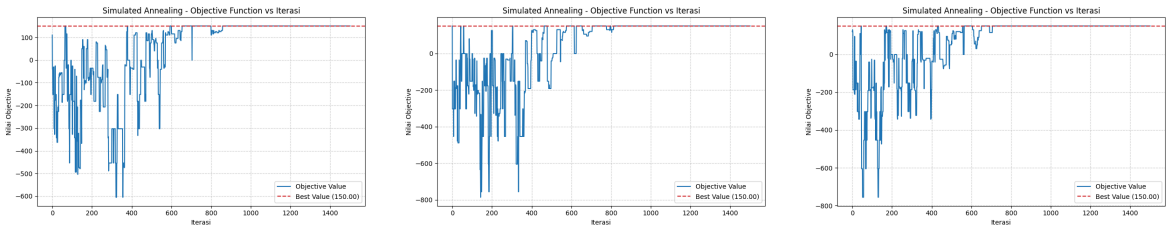
2.7.2 Simulated Annealing

Run	Objective Awal	Objective Akhir	Banyak Iterasi	Durasi (s)	Frekuensi Stuck
1	110	150	1500	0,1047	0,3600 (540/1500)
2	150	150	1500	0,0863	0,3407 (511/1500)
3	120	150	1500	0,0817	0,3280 (492/1500)

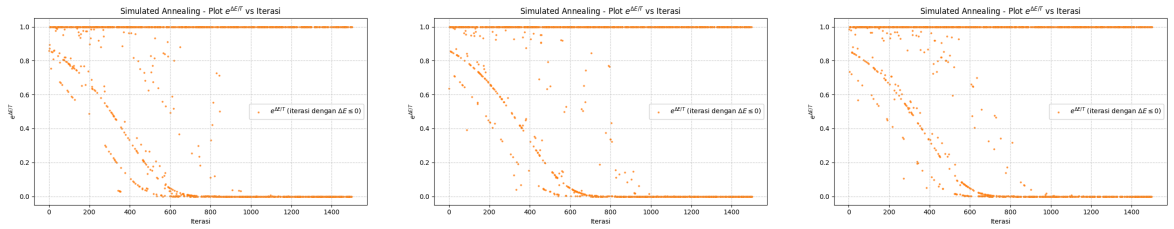
Tabel 2.3 Hasil eksperimen Simulated Annealing

Berdasarkan Tabel 2.3, ketiga pengujian Simulated Annealing menghasilkan nilai *objective function* akhir yang sama, yaitu 150, meskipun memiliki nilai awal yang berbeda. Pada pengujian pertama, nilai objektif meningkat dari 110 menjadi 150, pengujian kedua mempertahankan nilai 150, sedangkan pengujian ketiga meningkat dari 120 menjadi 150. Seluruh pengujian dilakukan selama 1500 iterasi dengan durasi masing-masing 0,1047 detik, 0,0863 detik, dan 0,0817 detik.

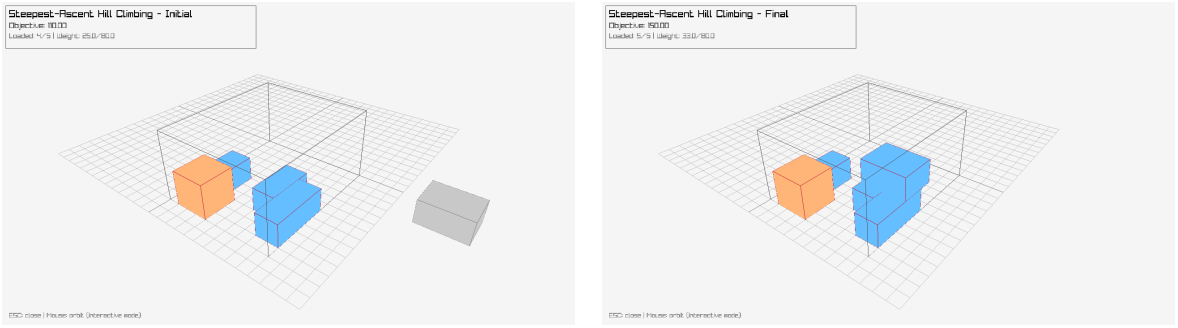
Frekuensi *stuck* yang tercatat pada ketiga pengujian berturut-turut adalah 0,3600 (540 dari 1500 iterasi), 0,3407 (511 dari 1500 iterasi), dan 0,3280 (492 dari 1500 iterasi). Seluruh pengujian menghasilkan konfigurasi akhir yang valid, dengan lima paket berhasil dimuat dan total bobot 33 dari kapasitas maksimum 80. Hasil ini menunjukkan Simulated Annealing mampu menghasilkan nilai objektif akhir yang konsisten pada dataset yang digunakan.



Gambar 2.4 Plot objective function terhadap iterasi (Simulated Annealing)



Gambar 2.5 Plot e^(ΔE/T) terhadap iterasi



Gambar 2.6 State awal dan akhir Simulated Annealing

2.7.3 Genetic Algorithm

a. Jumlah populasi sebagai kontrol

Iterasi	Run	Objective Akhir	Durasi (s)
50	1	150	0,1316
50	2	150	0,1127
50	3	150	0,1322
100	1	150	0,2509
100	2	150	0,2476
100	3	150	0,2356
200	1	150	0,4602
200	2	150	0,5080
200	3	150	0,4492

Tabel 2.4 Hasil GA dengan variasi banyak iterasi

Pada eksperimen pertama, jumlah populasi ditetapkan sebesar 40 individu, sedangkan banyak iterasi divariasikan menjadi 50, 100, dan 200. Setiap konfigurasi dijalankan sebanyak tiga kali untuk mengamati pengaruh jumlah iterasi terhadap nilai objektif dan durasi pencarian.

Berdasarkan Tabel 2.4, seluruh pengujian menghasilkan nilai objektif akhir sebesar 150 dengan konfigurasi yang valid. Rata-rata durasi pencarian untuk 50 iterasi adalah 0,1255 detik, untuk 100 iterasi sebesar 0,2447 detik, dan untuk 200 iterasi sebesar 0,4725 detik. Pada dataset yang diuji, konfigurasi 50 iterasi memberikan durasi pencarian paling singkat.

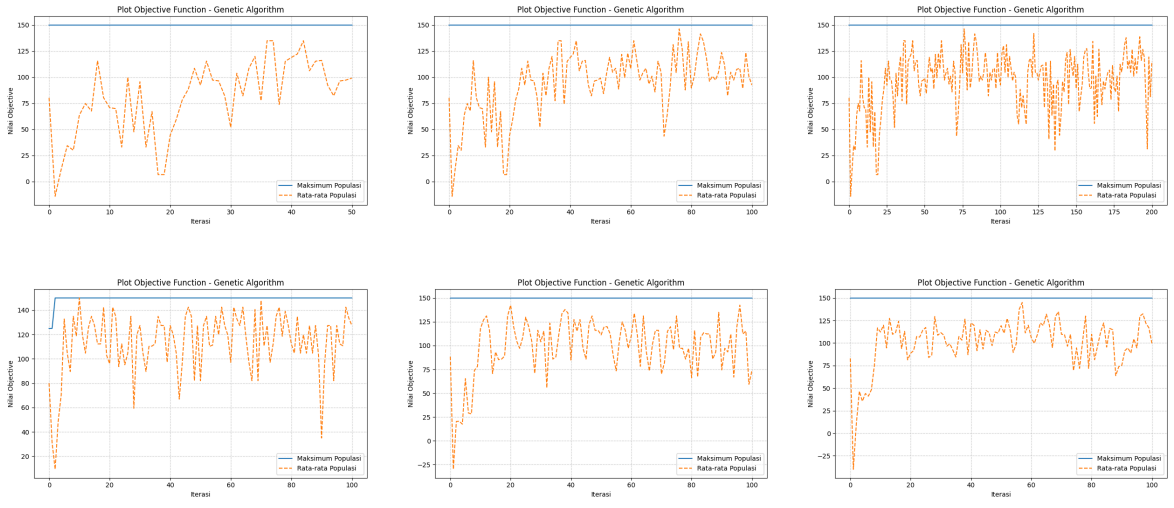
b. Banyak iterasi sebagai kontrol

Populasi	Run	Objective Akhir	Durasi (s)
20	1	150	0,1178
20	2	150	0,1196
20	3	150	0,1117
40	1	150	0,2509
40	2	150	0,2476
40	3	150	0,2356
60	1	150	0,3427
60	2	150	0,4005
60	3	150	0,3297

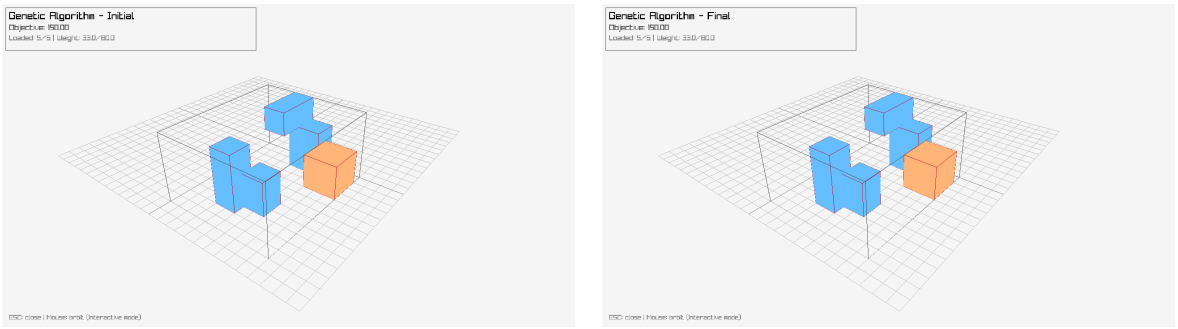
Tabel 2.5 Hasil GA dengan variasi jumlah populasi

Pada eksperimen kedua, banyak iterasi ditetapkan sebesar 100 iterasi, sedangkan ukuran populasi divariasikan menjadi 20, 40, dan 60 individu. Setiap konfigurasi dijalankan sebanyak tiga kali untuk mengetahui pengaruh jumlah populasi terhadap hasil optimasi dan durasi komputasi.

Berdasarkan Tabel 2.5, seluruh pengujian menghasilkan nilai objektif akhir sebesar 150 dengan konfigurasi yang valid. Rata-rata durasi pencarian untuk populasi 20 adalah 0,1164 detik, untuk populasi 40 sebesar 0,2447 detik, dan untuk populasi 60 sebesar 0,3576 detik. Populasi 20 memberikan waktu pencarian paling singkat dalam rentang parameter yang diuji.



Gambar 2.7 Plot objective maksimum dan rata-rata populasi terhadap iterasi (per konfigurasi)



Gambar 2.8 State awal dan akhir Genetic Algorithm

2.8 Analisis Hasil Eksperimen

2.8.1 Kedekatan terhadap Global Optima

Berdasarkan hasil eksperimen sebelumnya, ditemukan bahwa seluruh pengujian Steepest-ascent Hill-climbing, Simulated Annealing, dan Genetic Algorithm menghasilkan nilai objective function akhir sebesar 150. Nilai tersebut menunjukkan bahwa algoritma-algoritma tersebut mampu menemukan konfigurasi pemuatan dengan kualitas yang sama pada dataset yang digunakan.

Pada konfigurasi akhir, seluruh paket berhasil dimuat ke dalam peti kemas dengan total berat 33 dari kapasitas maksimum 80 tanpa melanggar batasan pemuatan. Sehingga nilai objective function sebesar 150 merupakan nilai terbaik yang dapat ditemukan ketiga algoritma selama ujian

Meskipun demikian, kesamaan nilai objektif akhir belum membuktikan bahwa solusi tersebut merupakan global optimum. Pembuktian tersebut memerlukan perbandingan dengan metode pencarian yang dapat menjamin optimalitas. Oleh karena itu, nilai 150 dalam eksperimen ini lebih tepat disebut sebagai nilai objektif terbaik yang ditemukan.

2.8.2 Perbandingan Hasil Antar Algoritma

Berdasarkan Tabel 2.2, Tabel 2.3, dan Tabel 2.4 pada Bagian 2.7, ketiga algoritma menghasilkan nilai objektif akhir yang konsisten, yaitu 150. Perbedaan utama terletak pada mekanisme pencarian dan jumlah iterasi yang diperlukan untuk mencapai nilai tersebut.

Steepest-ascent Hill-climbing mencapai nilai objektif 150 dalam 2–4 iterasi. Algoritma ini memilih neighbor terbaik pada setiap iterasi dan menghentikan pencarian ketika tidak ditemukan neighbor yang memberikan peningkatan nilai objektif. Pendekatan tersebut menghasilkan waktu pencarian yang singkat, tetapi pencarian dapat berhenti pada local optimum.

Simulated Annealing mencapai nilai objektif yang sama setelah 1.500 iterasi pada setiap pengujian. Algoritma ini memungkinkan penerimaan neighbor yang memiliki nilai objektif lebih rendah berdasarkan probabilitas yang dipengaruhi temperatur. Mekanisme tersebut memberikan peluang untuk keluar dari local optimum, meskipun pada dataset ini tidak menghasilkan nilai akhir yang lebih tinggi dibandingkan Hill-climbing.

Genetic Algorithm menghasilkan nilai objektif akhir 150 pada seluruh konfigurasi jumlah iterasi dan ukuran populasi yang diuji. Algoritma ini melakukan pencarian melalui populasi individu, seleksi orang tua, crossover, dan mutasi. Hasil tersebut menunjukkan bahwa GA mampu menemukan solusi dengan kualitas yang sama pada berbagai konfigurasi parameter yang diuji.

Dengan demikian, tidak terdapat perbedaan kualitas solusi akhir yang teramati di antara ketiga algoritma pada dataset ini. Perbandingan keunggulan algoritma selanjutnya perlu mempertimbangkan durasi komputasi, konsistensi hasil, dan pengaruh parameter pencarian.

2.8.3 Perbandingan Durasi

Berdasarkan hasil eksperimen pada Bagian 2.7, Steepest-ascent Hill-climbing memiliki durasi pencarian rata-rata sekitar 0,0043 detik. Simulated Annealing memiliki durasi rata-rata sekitar 0,0566 detik, sedangkan Genetic Algorithm menunjukkan durasi yang bergantung pada jumlah iterasi dan ukuran populasinya.

Pada pengujian Hill-climbing, durasi berada pada rentang 0,0039–0,0051 detik. Simulated Annealing membutuhkan waktu 0,0499–0,0643 detik untuk menyelesaikan 1.500 iterasi. Hasil tersebut menunjukkan bahwa Hill-climbing memiliki waktu pencarian paling singkat pada konfigurasi pengujian yang digunakan.

Pada Genetic Algorithm, rata-rata durasi untuk 50 iterasi adalah 0,0722 detik, untuk 100 iterasi adalah 0,1798 detik, dan untuk 200 iterasi adalah 0,3215 detik. Pada pengujian variasi populasi dengan 100 iterasi, rata-rata durasi untuk populasi 20, 40, dan 60 individu masing-masing sekitar 0,0661 detik, 0,1483 detik, dan 0,2264 detik.

Perbedaan durasi tersebut berkaitan dengan banyaknya evaluasi objective function yang dilakukan selama pencarian. Peningkatan jumlah iterasi memperpanjang proses evolusi, sedangkan peningkatan ukuran populasi menambah jumlah individu yang perlu dievaluasi pada setiap generasi.

Namun, perbandingan durasi antar algoritma perlu dilakukan secara hati-hati karena jumlah iterasi, ukuran populasi, dan mekanisme pencarian masing-masing berbeda. Dengan demikian, hasil eksperimen menunjukkan bahwa Hill-climbing paling cepat pada konfigurasi yang diuji, tetapi belum cukup untuk menyimpulkan bahwa algoritma tersebut selalu paling efisien untuk seluruh ukuran persoalan.

2.8.4 Konsistensi Hasil

Ketiga algoritma menunjukkan konsistensi nilai objektif akhir yang tinggi pada pengujian yang dilakukan. Steepest-ascent Hill-climbing menghasilkan nilai 150 pada tiga random seed, yaitu 42, 43, dan 44. Simulated Annealing juga menghasilkan nilai 150 pada ketiga random seed tersebut.

Genetic Algorithm menghasilkan nilai objektif akhir 150 pada seluruh pengujian variasi iterasi dan ukuran populasi yang tercantum pada Tabel 2.4 dan Tabel 2.5. Dengan demikian, perubahan parameter yang diuji tidak mengubah kualitas solusi akhir pada dataset ini.

Meskipun demikian, konsistensi tersebut hanya berlaku pada pengujian dan dataset yang digunakan. Dataset yang relatif kecil, dengan lima paket, mungkin belum cukup kompleks untuk memperlihatkan perbedaan kemampuan eksplorasi masing-masing algoritma.

Pengujian pada dataset yang lebih besar dan beragam diperlukan untuk menilai konsistensi algoritma secara lebih menyeluruh.

2.8.5 Pengaruh Iterasi dan Populasi pada Genetic Algorithm

Eksperimen Genetic Algorithm dilakukan dengan memvariasikan jumlah iterasi dan ukuran populasi untuk mengamati pengaruh kedua parameter terhadap kualitas solusi dan durasi pencarian.

Pada pengujian variasi iterasi, jumlah populasi ditetapkan sebesar 40 individu, sedangkan jumlah iterasi divariasikan menjadi 50, 100, dan 200. Seluruh konfigurasi menghasilkan nilai objektif akhir 150. Akan tetapi, rata-rata durasi meningkat dari 0,0722 detik pada 50 iterasi menjadi 0,1798 detik pada 100 iterasi dan 0,3215 detik pada 200 iterasi. Hal ini menunjukkan bahwa penambahan iterasi meningkatkan biaya komputasi tanpa memberikan peningkatan kualitas solusi pada dataset yang digunakan.

Pada pengujian variasi populasi, jumlah iterasi ditetapkan sebesar 100, sedangkan ukuran populasi divariasikan menjadi 20, 40, dan 60 individu. Seluruh konfigurasi juga menghasilkan nilai objektif akhir 150. Rata-rata durasi meningkat dari sekitar 0,0661 detik untuk populasi 20 individu menjadi 0,1483 detik untuk populasi 40 individu dan 0,2264 detik untuk populasi 60 individu.

Peningkatan ukuran populasi memungkinkan algoritma mempertimbangkan lebih banyak kandidat solusi pada setiap generasi, tetapi juga meningkatkan jumlah evaluasi fitness. Pada eksperimen ini, peningkatan tersebut tidak menghasilkan nilai objektif akhir yang lebih tinggi.

Berdasarkan kedua pengujian, konfigurasi 40 individu dengan 50 iterasi merupakan pilihan yang paling efisien di antara variasi jumlah iterasi yang diuji karena menghasilkan nilai objektif akhir yang sama dengan durasi rata-rata paling rendah. Sementara itu, konfigurasi 20 individu dengan 100 iterasi memiliki durasi rata-rata terendah pada pengujian variasi ukuran populasi. Pemilihan konfigurasi terbaik tetap bergantung pada kebutuhan komputasi dan karakteristik dataset yang digunakan.

2.8.6 Analisis Tambahan

Hasil eksperimen memperlihatkan bahwa nilai objektif awal memengaruhi besarnya peningkatan yang diamati pada algoritma yang menggunakan satu state awal. Pada Steepest-ascent Hill-climbing, misalnya, nilai objektif meningkat dari -250 menjadi 150 pada pengujian pertama, dari -485 menjadi 150 pada pengujian kedua, dan dari 75 menjadi 150 pada pengujian ketiga. Simulated Annealing menunjukkan pola nilai awal dan akhir yang sama pada ketiga random seed.

Perbedaan nilai awal tersebut menunjukkan bahwa konfigurasi awal hasil pembangkitan acak dapat memiliki kualitas yang berbeda. Walaupun demikian, kedua algoritma berhasil mencapai nilai objektif akhir yang sama pada seluruh pengujian. Hal ini mengindikasikan bahwa mekanisme pencarian yang digunakan cukup efektif untuk memperbaiki konfigurasi awal pada dataset ini.

Pada Simulated Annealing, frekuensi stuck tercatat sebesar 0,4233, 0,4087, dan 0,4040 untuk masing-masing pengujian. Frekuensi tersebut menunjukkan bahwa algoritma cukup sering

mengalami iterasi tanpa peningkatan nilai objektif yang signifikan. Namun, kondisi tersebut tidak menghalangi algoritma untuk mencapai nilai akhir 150. Mekanisme penerimaan solusi berdasarkan temperatur memungkinkan pencarian tetap berlangsung meskipun tidak terjadi peningkatan objektif secara langsung.

Secara keseluruhan, eksperimen menunjukkan bahwa ketiga algoritma mampu menghasilkan konfigurasi pemuatan yang valid dengan nilai objektif akhir yang sama. Hill-climbing unggul dalam durasi pencarian pada konfigurasi yang diuji, Simulated Annealing menyediakan mekanisme untuk menerima solusi yang lebih buruk sementara, sedangkan Genetic Algorithm memungkinkan eksplorasi melalui populasi kandidat solusi. Untuk memperoleh kesimpulan yang lebih kuat mengenai keunggulan masing-masing algoritma, eksperimen lanjutan perlu menggunakan dataset yang lebih besar, lebih banyak random seed, dan kondisi pengujian yang sebanding.

BAB III KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan implementasi dan hasil eksperimen yang telah dilakukan, dapat disimpulkan beberapa hal sebagai berikut:

1. Permasalahan Optimasi pemuatan paket ke dalam peti kemas truk berhasil dimodelkan sebagai permasalahan optimasi yang mempertimbangkan simensi, orientasi, posisi, nilai, dan berat paket. Representasi state, fungsi objektif dengan penalti pelanggaran, serta mekanisme pembangkitan *neighbor* memungkinkan algoritma pencarian mengevaluasi dan memperbaiki konfigurasi pemuatan paket.
2. Dari algortima yang telah diimplementasikan, semuanya berhasil menghasilkan nilai objective function akhir sebesar 150 pada seluruh pengujian yang dilakukan. Konfigurasi akhir tersebut memenuhi batasan pemuatan dengan lima paket berhasil dimuat dan total berat sebesar 33 dari kapasitas maksimum 80. Meskipun demikian, hasil tersebut menunjukkan solusi terbaik yang ditemukan dalam eksperimen dan belum membuktikan bahwa nilai 150 merupakan global optimum.
3. Steepest-ascent Hill-climbing menunjukkan waktu pencarian paling singkat di antara konfigurasi yang diuji, dengan durasi rata-rata sekitar 0,0043 detik. Algoritma ini mencapai solusi akhir dalam 2–4 iterasi dengan memilih neighbor terbaik pada setiap langkah. Sementara itu, Simulated Annealing membutuhkan rata-rata sekitar 0,0566 detik untuk 1.500 iterasi. Perbedaan ini menunjukkan bahwa Hill-climbing lebih cepat pada dataset yang digunakan, sedangkan Simulated Annealing menyediakan mekanisme penerimaan solusi yang lebih buruk untuk membantu pencarian keluar dari local optimum.
4. Pada Genetic Algorithm, penambahan jumlah iterasi dan ukuran populasi meningkatkan durasi komputasi, tetapi tidak meningkatkan nilai objektif akhir pada dataset yang diuji. Variasi 50, 100, dan 200 iterasi dengan populasi 40 individu seluruhnya menghasilkan nilai objektif 150. Demikian pula, variasi populasi 20, 40, dan 60 individu dengan 100 iterasi menghasilkan nilai akhir yang sama. Oleh karena itu, penambahan sumber daya komputasi tidak selalu memberikan peningkatan kualitas solusi, terutama pada persoalan dengan ukuran dan karakteristik dataset yang relatif kecil.
5. Secara keseluruhan, algoritma yang telah diimplementasikan mampu menemukan konfigurasi pemuatan yang valid dengan kualitas solusi yang sama pada dataset

eksperimen. Perbedaan utama terletak pada mekanisme pencarian dan waktu komputasi. Hill-climbing sesuai untuk pencarian cepat pada persoalan yang diuji, Simulated Annealing memberikan fleksibilitas dalam eksplorasi ruang solusi, sedangkan Genetic Algorithm memungkinkan eksplorasi melalui populasi kandidat solusi. Keunggulan masing-masing algoritma pada persoalan yang lebih kompleks masih perlu dievaluasi melalui eksperimen lanjutan.

3.2 Saran

Berdasarkan implementasi dan hasil eksperimen, terdapat beberapa saran untuk pengembangan dan evaluasi implementasi algoritma local search selanjutnya.

1. Menggunakan dataaset yang lebih besar dan beragamm
2. Melakukan pengujian dengan lebih banyak random seed
3. Melakukan penyetelan parameter algoritma
4. Meningkatkan mekanisme pembangkitan solusi pada Genetic Algorithm
5. Mengoptimalkan implementasi dan visualisasi

Dengan pengembangan tersebut, kami berharap untuk dapat menghasilkan solusi pemuatan paket yang semakin baik, mempertahankan validitas konfigurasi, dan memberikan perbandingan kinerja algoritma yang lebih kuat pada berbagai karakteristik persoalan.

BAB IV PEMBAGIAN TUGAS

NIM	Nama	Tugas
13524143	Daniel Putra Rywandi S	Core, Hill-climbing, simulated annealing, bonus
13524138	Ahmad Rinofaros M	Models, Visualization, Loader
13524114	Mirza Tsabita W	Genetic Algorithm

Tabel 4.1 Pembagian tugas anggota kelompok

REFERENSI

1. Hoos, H.H. and Stutzle, T. (2005) Stochastic Local Search: Foundations and Applications, Morgan Kaufmann.
2. Python Documentation
3. Introduction to Hill Climbing
4. Simulated Annealing Basics
5. Introduction to Genetic Algorithm
6. Pyray Basics

LAMPIRAN

Lampiran A. Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:
Nama/NIM :

1. Daniel Putra Rywandi S/13524143
2. Ahmad Rinofaros M/13524138
3. Mirza Tsabita W/13524114

Fakultas/Sekolah : Teknik Informatika/STEI-Komputasi
Kode Mata Kuliah : IF3170
Nama Mata Kuliah : Inteligensi Artifisial
Judul Tugas/Proposal : Tugas Besar 1

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	*No AI*
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	*No AI*
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	*AI-assisted workflow (Syntax dependency raylib sulit digunakan pada awal pembuatan visualisasi dan error dievaluasi menggunakan bantuan LLM)*
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	*No AI*

Lainnya, Jelaskan:		

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 20 Oktober 2026



Ahmad Rinofaros M

13524138

Lampiran B. Tautan Repository

Repository: https://github.com/SpicyCoinCracker/IF3170_G58_ApertureScience